Doc. no.:	P1957R0
Date:	2019-11-04
Audience:	LWG, CWG
Reply-to:	Zhihao Yuan <zy at miator dot net>

Converting from `T*` to `bool` should be considered narrowing (re: US 212)

Background

LWG 3228 (https://cplusplus.github.io/LWG/issue3228) "Surprising `variant` construction" shows that, after applying P0608R3[1], a user-defined type that is convertible to `bool` now may construct an alternative type other than `bool`:

```
bitset<4> b("0101");
variant<bool, int> v = b[1];  // holds int
```

This is a direct result of a workaround introduced in R3 of the paper. The original issue found in R1 was that treating boolean conversion as narrowing conversion by detecting it is not implementable in the library. And the workaround was to ban conversions to `bool` naively.

Proposal

I propose to drop the workaround and treat a conversion from a pointer type or a pointer-to-member type to `bool` as narrowing conversion in the core language.

Discussion

If MSVC standard library implements P0608 today (libc++ and libstdc++ have shipped P0608R3) without special treatment of conversions to `bool` (equivalent to applying the proposed wording Part 2), we will end up with an ideal situation.

- `char*` argument cannot construct `bool` alternative,
- a user-defined type that is convertible to `char*` cannot construct `bool` alternative, and

- a user-defined type that is convertible to `bool` can construct a `bool` alternative.

Because MSVC considers non-literal pointer-to- `bool` conversion as a narrowing conversion:

```
// error C2397: conversion from 'char *' to 'bool' requires a narrowing conversion
bool y {new char()};
```

The rationale has been fully stated in N2215[2] when narrowing conversion was introduced to the standard in 2007:

> Some implicit casts, […] Others, such as **double->char** and **int*->bool**, are widely considered embarrassments.

> Our basic definition of narrowing (expressed using **decltype**) is that a conversion from **T** to **T2** is narrowing unless we can guarantee that
>
> ```
> T a = x;
> T2 b = a;
> T c = b;
> ```
>
> implies that **a==c** for every initial value **x**; that is, that the **T** to **T2** to **T** conversions are not value preserving.

One might argue that substituting in `T = char*` and `T2 = bool` will cause the 3rd line to fail to compile, which may suggest pointer-to- `bool` conversion to be slightly safer. Here is my response:

First, whether `T2` to `T` conversion is implicit or explicit does not change the fact that `T` to `T2` is not value preserving.

Second, when expressing what is not considered narrowing,

> […] or any combination of these conversions, except in cases where the initializer is a constant expression and the actual value will fit into the target object; that is, where **decltype(x)(T{x})==decltype(x)(x)**.

the wording also used casts.

Data and impact

When Google adopted P0608 in their codebase, Eric Fiselier found that constructing a `bool` alternative from a pointer argument is wrong 100% of the time. So I suspect that constructing a `bool` member from an *initializer-clause* in aggregate initialization is wrong 100% of the time:

```
struct A { bool x; int y; };
A a = { "file not found", ENOENT };
```

The impact of this change is no greater than adding narrowing conversion to C++11 and should be practical, given MSVC's implementation experience.

Constant evaluation

There is a question of whether narrowing conversion should exclude some cases when pointers are converting to `bool` in a constant expression. According to N2215's rationale, only `(T*)nullptr` to `bool` is value preserving, but this is not what MSVC implemented. Rather than evaluating constant expressions, MSVC deems only literals to be non-narrowing.

I think neither tweak is necessary. The cases we may allow are at best as meaningful as

```
int x = {1.0};
```

About `nullptr`

"Conversion" from `std::nullptr_t` to `bool` is not involved in the discussion on narrowing conversions. It is not a boolean conversion, because `nullptr` is a *null pointer constant*, but not a *null pointer value* or *null member pointer value*. In short, `std::nullptr_t` is not convertible to `bool`, but `bool` is constructible from `std::nullptr_t`:

```
// error: converting to 'bool' from 'std::nullptr_t' requires direct-initializatio
bool x = nullptr;
bool y = {nullptr};    // ditto
bool z{nullptr};       // ok
```

GCC implemented these correctly.

Implementation

https://reviews.llvm.org/D64034 (https://reviews.llvm.org/D64034)

Wording

The wording is relative to N4835.

Part 1

Modify 9.4.4 [dcl.init.list]/7 as follows:

> A narrowing conversion is an implicit conversion
>
> — from a floating-point type to an integer type, or
>
> […]
>
> — from an integer type or unscoped enumeration type to an integer type that cannot represent all the values of the original type, except where the source is a constant expression whose value after integral promotions will fit into the target type~~.~~, or
>
> — from a pointer type or a pointer-to-member type to `bool`.
>
> *[Note: … — end note] [Example:*
>
> ```
>  ...
> int ii = {2.0}; // error: narrows
> float f1 { x }; // error: might narrow
> float f2 { 7 }; // OK: 7 can be exactly represented as a float
> bool b = {"meow"}; // error: narrows
> int f(int);
> int a[] = { 2, f(2), f(2.0) }; // OK: the double-to-int conversion is not at
> ```
>
> *— end example]*

Add a new paragraph to C.5.4 [diff.cpp17.dcl.dcl]:

> **Affected subclause:** 9.4.4
>
> **Change:** Boolean conversion from pointer or pointer-to-member type is narrowing conversion.
>
> **Rationale:** Catches bugs.
>
> **Effect on original feature:** Valid C++ 2017 code may fail to compile in this International Standard. For example:
>
> ```
> bool y[] = { "bc" };  // ill-formed; previously well-formed
> ```

Part 2

Modify 20.7.3.1 [variant.ctor]/12 as indicated:

```
template<class T> constexpr variant(T&& t) noexcept(see below );
```

> Let T_j be a type that is determined as follows: build an imaginary function $FUN(T_i)$ for each alternative type T_i for which `T_i x[] = {std::forward<T>(t)};` is well-formed for some invented variable `x` ~~and, if `T_i` is *cv* `bool`, `remove_cvref_t<T>` is `bool`~~. […]

Modify 20.7.3.3 [variant.assign]/10 as indicated:

```
template<class T> variant& operator=(T&& t) noexcept(see below );
```

> Let T_j be a type that is determined as follows: build an imaginary function $FUN(T_i)$ for each alternative type T_i for which `T_i x[] = {std::forward<T>(t)};` is well-formed for some invented variable `x` ~~and, if `T_i` is *cv* `bool`, `remove_cvref_t<T>` is `bool`~~. […]

Acknowledgments

Thank Eric Fiselier for his work and thoughts.

References

1. Yuan, Zhihao. P0608R3 *A sane variant converting constructor*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0608r3.html (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0608r3.html) ↩

2. Stroustrup, Bjarne and Gabriel Dos Reis. N2215 *Initializer lists (Rev. 3)*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2215.pdf (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2215.pdf) ↩